

การพัฒนา Model โดยใช้ Python และ Software Package สำหรับผู้พัฒนา Model

Module 6 - Regression (continued)

Asst. Prof. Dr. Santitham Prom-on

1. Load data

```
In [1]: !wget -O mtcars.csv https://www.dropbox.com/scl/fi/n0e5sbzbc1sf90n4kr72h/mtcars.
--2026-08-28 03:44:10-- https://www.dropbox.com/scl/fi/n0e5sbzbc1sf90n4kr72h/mtc
ars.csv?rlkey=66jca70510jnrj94eigxiwtpn
Resolving www.dropbox.com (www.dropbox.com)... 162.125.1.18, 2620:100:6016:18::a2
7d:112
Connecting to www.dropbox.com (www.dropbox.com)|162.125.1.18|:443... connected.
HTTP request sent, awaiting response... 302 Found
Location: https://uc2f065faeaa818ea4d1d5387cd1.dl.dropboxusercontent.com/cd/0/inl
ine/DHDWqVOVtkww--PDdoIs4qUIv9QyCOSf6C-eQ6daw9gZmSM8QySTDuWgxov7U1KgYruvBRSRWxuf9
ptua-y0vBv26pH1jMXFjDvSDMg4sWS3AlkBq43E-7QDQkcZLFd1wjE/file# [following]
--2026-08-28 03:44:10-- https://uc2f065faeaa818ea4d1d5387cd1.dl.dropboxuserconte
nt.com/cd/0/inline/DHDWqVOVtkww--PDdoIs4qUIv9QyCOSf6C-eQ6daw9gZmSM8QySTDuWgxov7U1
KgYruvBRSRWxuf9ptua-y0vBv26pH1jMXFjDvSDMg4sWS3AlkBq43E-7QDQkcZLFd1wjE/file
Resolving uc2f065faeaa818ea4d1d5387cd1.dl.dropboxusercontent.com (uc2f065faeaa818
ea4d1d5387cd1.dl.dropboxusercontent.com)... 162.125.1.15, 2620:100:6016:15::a27d:
10f
Connecting to uc2f065faeaa818ea4d1d5387cd1.dl.dropboxusercontent.com (uc2f065faea
a818ea4d1d5387cd1.dl.dropboxusercontent.com)|162.125.1.15|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 1726 (1.7K) [text/plain]
Saving to: 'mtcars.csv'

mtcars.csv          100%[=====>]    1.69K  --.-KB/s    in 0s

2026-08-28 03:44:11 (354 MB/s) - 'mtcars.csv' saved [1726/1726]
```

```
In [2]: import pandas as pd

mtcars = pd.read_csv('mtcars.csv', index_col=0)
mtcars.head()
```

Out[2]:

	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
Mazda RX4	21.0	6	160.0	110	3.90	2.620	16.46	0	1	4	4
Mazda RX4 Wag	21.0	6	160.0	110	3.90	2.875	17.02	0	1	4	4
Datsun 710	22.8	4	108.0	93	3.85	2.320	18.61	1	1	4	1
Hornet 4 Drive	21.4	6	258.0	110	3.08	3.215	19.44	1	0	3	1
Hornet Sportabout	18.7	8	360.0	175	3.15	3.440	17.02	0	0	3	2

2. Assess correlations between variables

As a correlation matrix

In [3]: `mtcars.corr()`

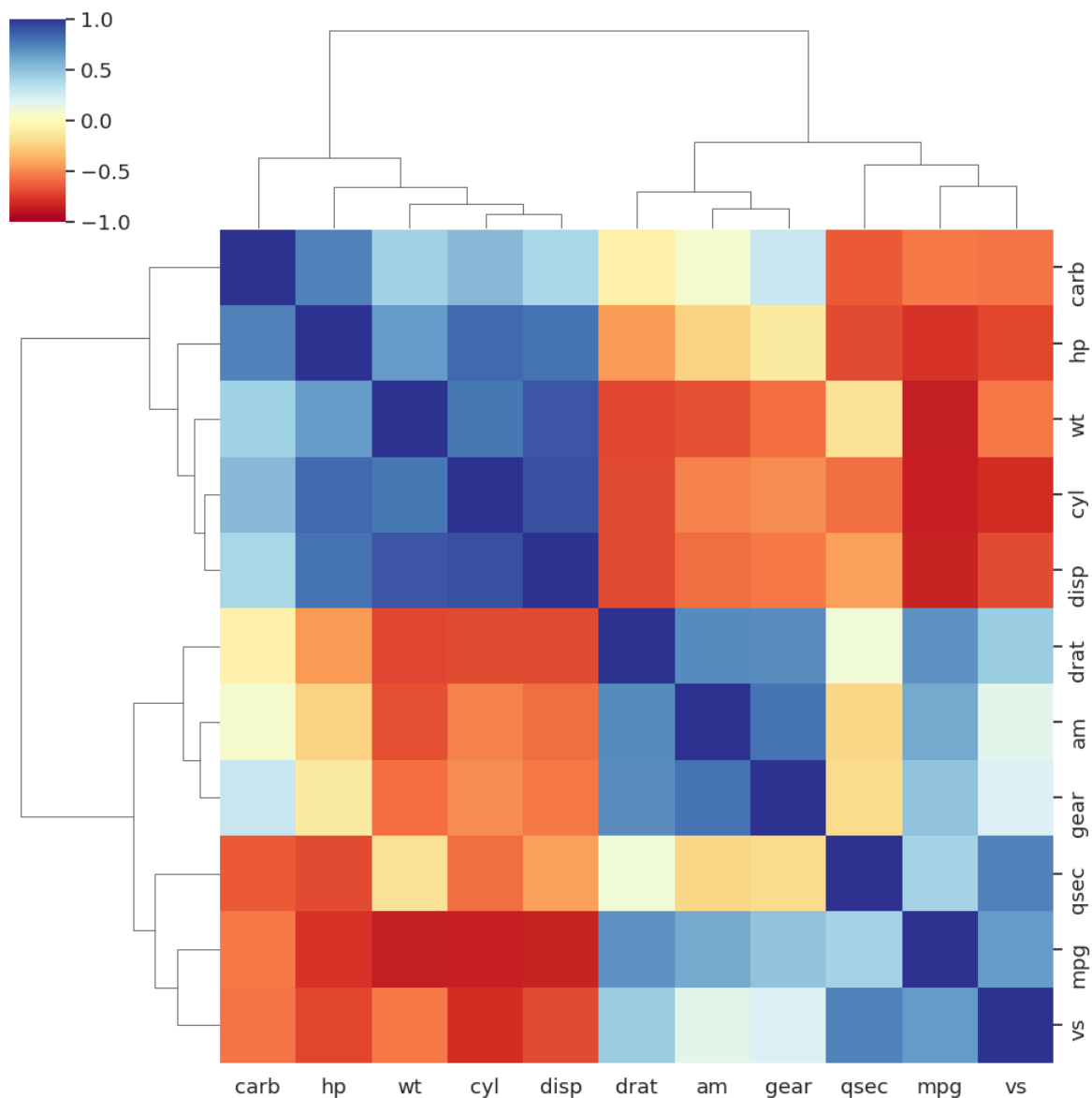
Out[3]:

	mpg	cyl	disp	hp	drat	wt	qsec
mpg	1.000000	-0.852162	-0.847551	-0.776168	0.681172	-0.867659	0.418684
cyl	-0.852162	1.000000	0.902033	0.832447	-0.699938	0.782496	-0.591242
disp	-0.847551	0.902033	1.000000	0.790949	-0.710214	0.887980	-0.433698
hp	-0.776168	0.832447	0.790949	1.000000	-0.448759	0.658748	-0.708223
drat	0.681172	-0.699938	-0.710214	-0.448759	1.000000	-0.712441	0.091205
wt	-0.867659	0.782496	0.887980	0.658748	-0.712441	1.000000	-0.174716
qsec	0.418684	-0.591242	-0.433698	-0.708223	0.091205	-0.174716	1.000000
vs	0.664039	-0.810812	-0.710416	-0.723097	0.440278	-0.554916	0.744535
am	0.599832	-0.522607	-0.591227	-0.243204	0.712711	-0.692495	-0.229861
gear	0.480285	-0.492687	-0.555569	-0.125704	0.699610	-0.583287	-0.212682
carb	-0.550925	0.526988	0.394977	0.749812	-0.090790	0.427606	-0.656249



As a heatmap

In [4]: `import seaborn as sns
import matplotlib.pyplot as plt
sns.set(font_scale=1.2)`In [5]: `sns.clustermap(mtcars.corr(), cmap='RdYlBu', vmin=-1, vmax=1)
plt.show()`

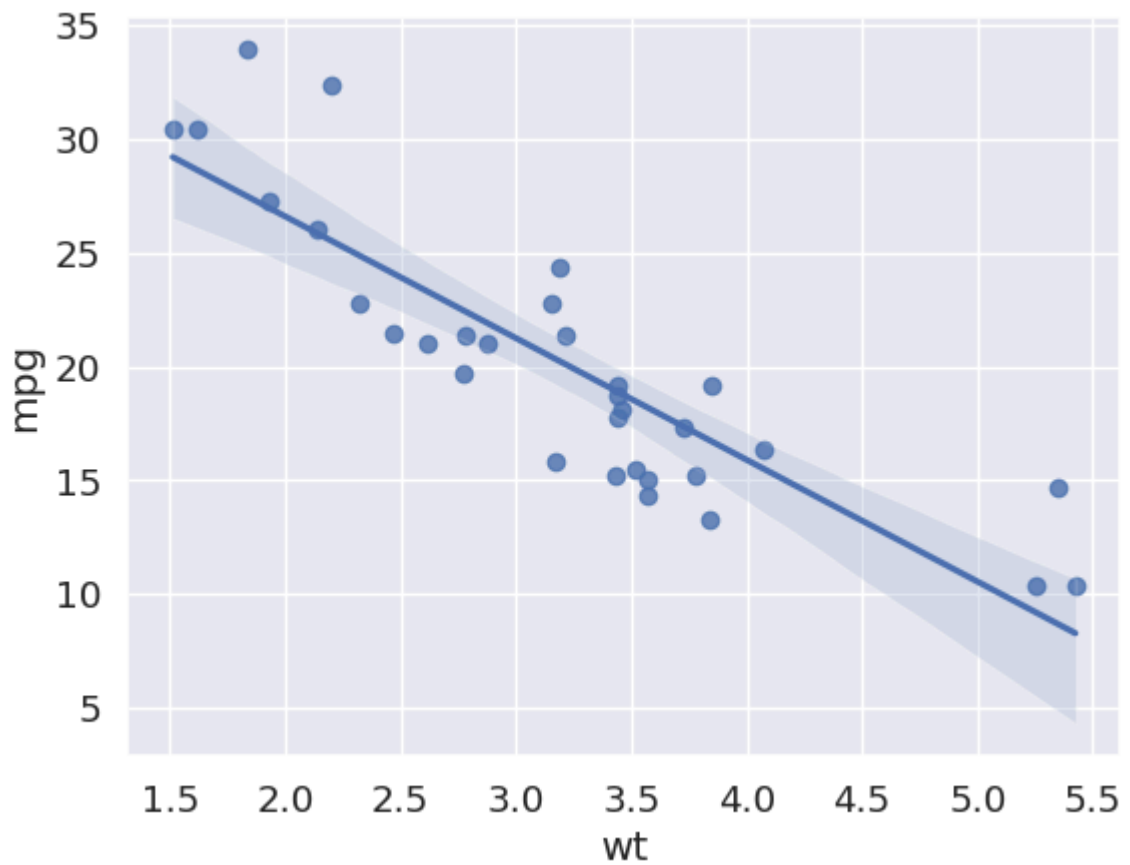


3. Visualize a linear relationship as a scatterplot

Negative relationship

```
In [6]: c = mtcars['mpg'].corr(mtcars['wt'])
print("Correlation = %0.2f"%mtcars['mpg'].corr(mtcars['wt']))
sns.regplot(data=mtcars, x='wt', y='mpg')
plt.show()
```

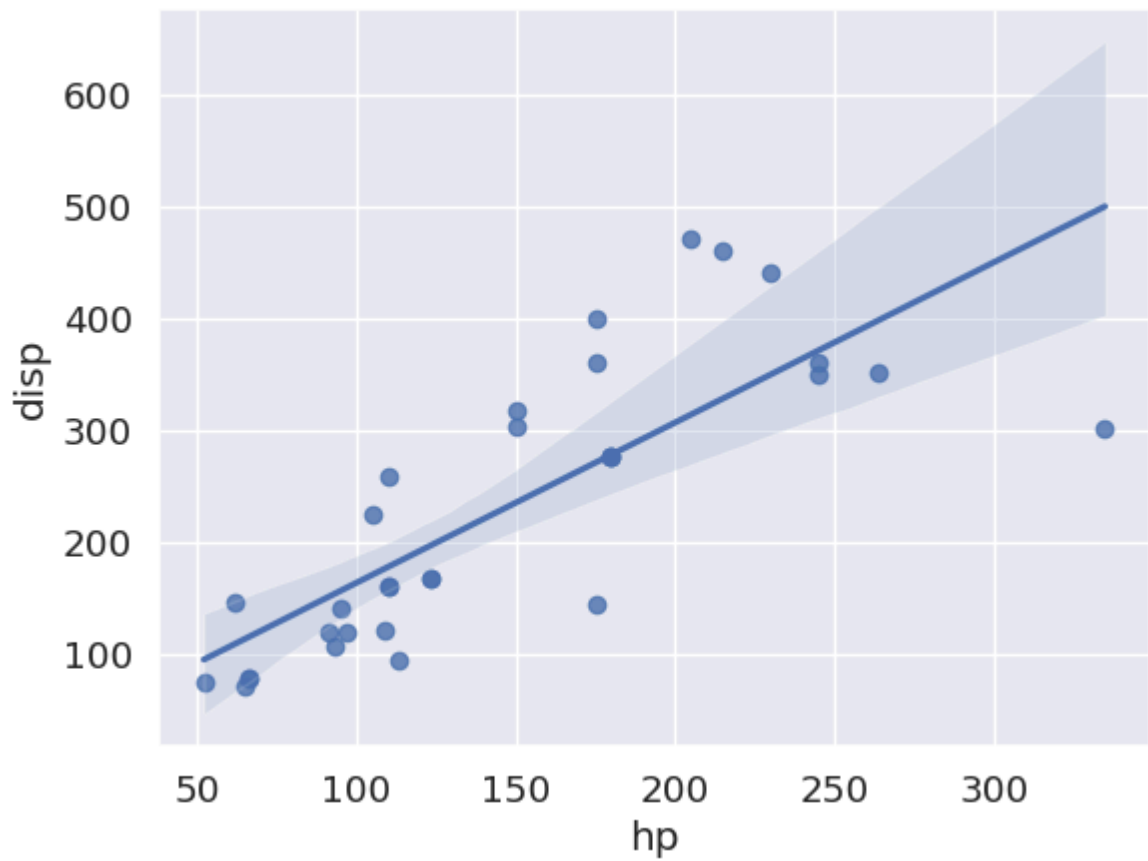
Correlation = -0.87



Positive relationship

```
In [7]: c = mtcars['hp'].corr(mtcars['disp'])  
print("Correlation = %0.2f"%mtcars['hp'].corr(mtcars['disp']))  
sns.regplot(data=mtcars, x='hp', y='disp')  
plt.show()
```

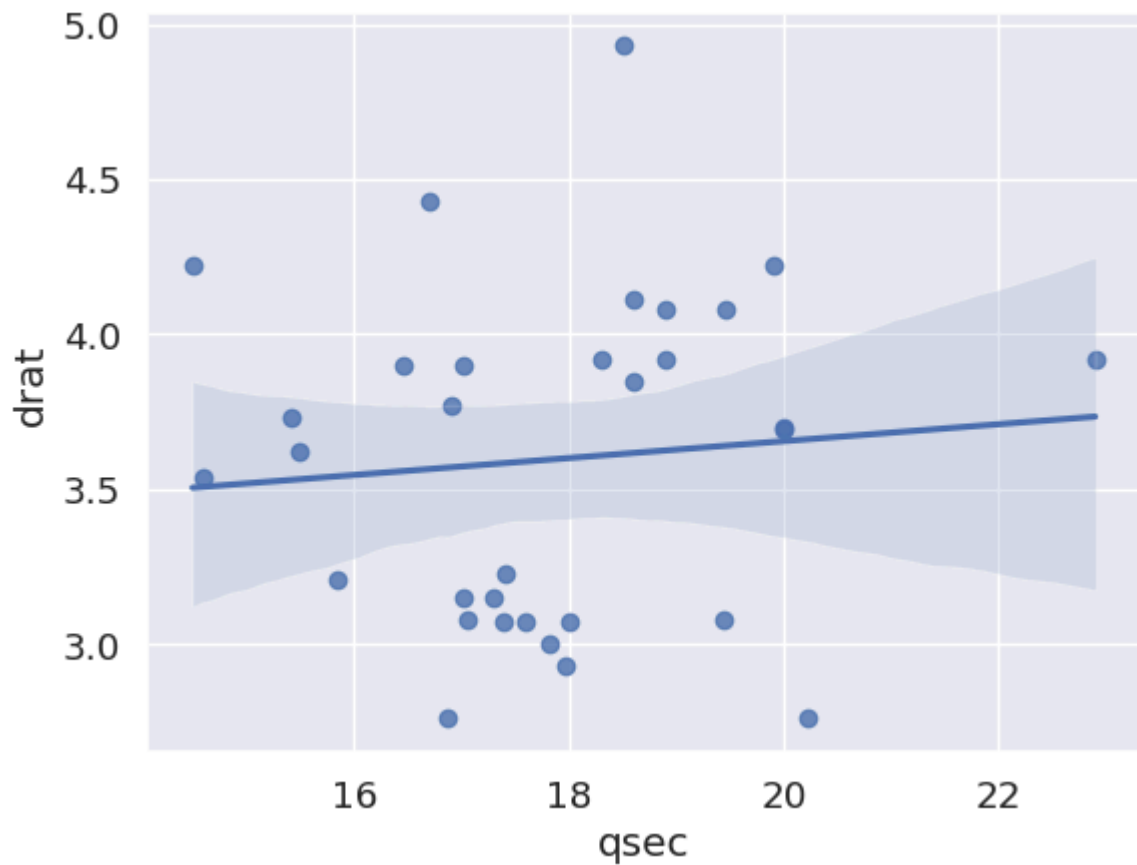
Correlation = 0.79



No relationship

```
In [8]: c = mtcars['qsec'].corr(mtcars['drat'])
print("Correlation = %0.2f"%mtcars['qsec'].corr(mtcars['drat']))
sns.regplot(data=mtcars, x='qsec', y='drat')
plt.show()
```

Correlation = 0.09



4. Linear regression

with statmodels

```
In [9]: from statsmodels.formula.api import ols

model = ols('mpg ~ wt', mtcars).fit()
model.summary()
```

Out[9]:

OLS Regression Results

Dep. Variable:	mpg	R-squared:	0.753
Model:	OLS	Adj. R-squared:	0.745
Method:	Least Squares	F-statistic:	91.38
Date:	Fri, 28 Aug 2026	Prob (F-statistic):	1.29e-10
Time:	03:44:30	Log-Likelihood:	-80.015
No. Observations:	32	AIC:	164.0
Df Residuals:	30	BIC:	167.0
Df Model:	1		
Covariance Type:	nonrobust		

	coef	std err	t	P> t	[0.025	0.975]
Intercept	37.2851	1.878	19.858	0.000	33.450	41.120
wt	-5.3445	0.559	-9.559	0.000	-6.486	-4.203

Omnibus: 2.988 **Durbin-Watson:** 1.252

Prob(Omnibus): 0.225 **Jarque-Bera (JB):** 2.399

Skew: 0.668 **Prob(JB):** 0.301

Kurtosis: 2.877 **Cond. No.** 12.7

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

In [10]: `model.predict(mtcars)`

Out[10]:

0

Mazda RX4	23.282611
Mazda RX4 Wag	21.919770
Datsun 710	24.885952
Hornet 4 Drive	20.102650
Hornet Sportabout	18.900144
Valiant	18.793255
Duster 360	18.205363
Merc 240D	20.236262
Merc 230	20.450041
Merc 280	18.900144
Merc 280C	18.900144
Merc 450SE	15.533127
Merc 450SL	17.350247
Merc 450SLC	17.083024
Cadillac Fleetwood	9.226650
Lincoln Continental	8.296712
Chrysler Imperial	8.718926
Fiat 128	25.527289
Honda Civic	28.653805
Toyota Corolla	27.478021
Toyota Corona	24.111004
Dodge Challenger	18.472586
AMC Javelin	18.926866
Camaro Z28	16.762355
Pontiac Firebird	16.735633
Fiat X1-9	26.943574
Porsche 914-2	25.847957
Lotus Europa	29.198941
Ford Pantera L	20.343151
Ferrari Dino	22.480940
Maserati Bora	18.205363
Volvo 142E	22.427495

dtype: float64

with sci-kit learn

```
In [11]: X = mtcars.drop(columns='mpg')  
y = mtcars['mpg']
```

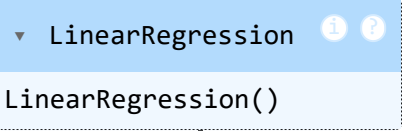
```
In [12]: from sklearn.preprocessing import StandardScaler  
scaler = StandardScaler()  
X_df = pd.DataFrame(scaler.fit_transform(X), columns = X.columns)
```

```
In [13]: X_df
```

Out[13]:

	cyl	disp	hp	drat	wt	qsec	vs	an
0	-0.106668	-0.579750	-0.543655	0.576594	-0.620167	-0.789601	-0.881917	1.20894
1	-0.106668	-0.579750	-0.543655	0.576594	-0.355382	-0.471202	-0.881917	1.20894
2	-1.244457	-1.006026	-0.795570	0.481584	-0.931678	0.432823	1.133893	1.20894
3	-0.106668	0.223615	-0.543655	-0.981576	-0.002336	0.904736	1.133893	-0.827170
4	1.031121	1.059772	0.419550	-0.848562	0.231297	-0.471202	-0.881917	-0.827170
5	-0.106668	-0.046906	-0.617748	-1.589643	0.252064	1.348220	1.133893	-0.827170
6	1.031121	1.059772	1.456847	-0.734549	0.366285	-1.142114	-0.881917	-0.827170
7	-1.244457	-0.688779	-1.254944	0.177551	-0.028296	1.223135	1.133893	-0.827170
8	-1.244457	-0.737144	-0.765933	0.614599	-0.069830	2.871986	1.133893	-0.827170
9	-0.106668	-0.517448	-0.351014	0.614599	0.231297	0.256567	1.133893	-0.827170
10	-0.106668	-0.517448	-0.351014	0.614599	0.231297	0.597708	1.133893	-0.827170
11	1.031121	0.369533	0.493642	-1.000578	0.885470	-0.255145	-0.881917	-0.827170
12	1.031121	0.369533	0.493642	-1.000578	0.532424	-0.141432	-0.881917	-0.827170
13	1.031121	0.369533	0.493642	-1.000578	0.584343	0.085996	-0.881917	-0.827170
14	1.031121	1.977904	0.864106	-1.266608	2.110747	0.074625	-0.881917	-0.827170
15	1.031121	1.879533	1.012291	-1.133593	2.291423	-0.016346	-0.881917	-0.827170
16	1.031121	1.715580	1.234569	-0.696545	2.209392	-0.243774	-0.881917	-0.827170
17	-1.244457	-1.246216	-1.195670	0.918632	-1.056282	0.921793	1.133893	1.20894
18	-1.244457	-1.270809	-1.403130	2.533809	-1.663729	0.381652	1.133893	1.20894
19	-1.244457	-1.308518	-1.210489	1.184661	-1.435287	1.166278	1.133893	1.20894
20	-1.244457	-0.906835	-0.736296	0.196553	-0.781114	1.228820	1.133893	-0.827170
21	1.031121	0.715472	0.049086	-1.589643	0.314367	-0.556487	-0.881917	-0.827170
22	1.031121	0.600705	0.049086	-0.848562	0.226105	-0.312002	-0.881917	-0.827170
23	1.031121	0.977795	1.456847	0.253559	0.646645	-1.386598	-0.881917	-0.827170
24	1.031121	1.387676	0.419550	-0.981576	0.651837	-0.454145	-0.881917	-0.827170
25	-1.244457	-1.243757	-1.195670	0.918632	-1.331450	0.597708	1.133893	1.20894
26	-1.244457	-0.905195	-0.825207	1.583705	-1.118584	-0.653144	-0.881917	1.20894
27	-1.244457	-1.111775	-0.499199	0.329567	-1.769642	-0.539430	1.133893	1.20894
28	1.031121	0.985993	1.738399	1.184661	-0.049063	-1.903996	-0.881917	1.20894
29	-0.106668	-0.702714	0.419550	0.044536	-0.464411	-1.335427	-0.881917	1.20894
30	1.031121	0.576113	2.790515	-0.107481	0.366285	-1.847139	-0.881917	1.20894
31	-1.244457	-0.899457	-0.558473	0.975638	-0.454027	0.427138	1.133893	1.20894

```
In [14]: from sklearn.linear_model import LinearRegression
lm = LinearRegression()
lm.fit(X_df,y)
```

Out[14]:  LinearRegression()

```
In [15]: from sklearn.metrics import r2_score
r2_score(y_pred=lm.predict(X_df), y_true=y)
```

Out[15]: 0.8690157644777647

```
In [16]: pd.DataFrame(dict(Feature = X_df.columns,
                          Coefficient = lm.coef_))
```

Out[16]:

	Feature	Coefficient
0	cyl	-0.195890
1	disp	1.626723
2	hp	-1.449679
3	drat	0.414224
4	wt	-3.578015
5	qsec	1.444047
6	vs	0.157635
7	am	1.237765
8	gear	0.475951
9	carb	-0.317029

5. Poisson regression

```
In [17]: !wget -O mines.csv https://www.dropbox.com/sc1/fi/y5wi9iqgyxjiwm4stdbx3/mines.cs
```

```
--2026-08-28 03:44:45-- https://www.dropbox.com/scl/fi/y5wi9iqgyxjiwm4stdbx3/min
es.csv?rlkey=t04s3l1ynyhr3moknm5y55ej8
Resolving www.dropbox.com (www.dropbox.com)... 162.125.1.18, 2620:100:6016:18::a2
7d:112
Connecting to www.dropbox.com (www.dropbox.com)|162.125.1.18|:443... connected.
HTTP request sent, awaiting response... 302 Found
Location: https://uc7aba254521e035c457b56bff3d.dl.dropboxusercontent.com/cd/0/inl
ine/DHDTBW0PaJtKmfFsk8gxq29FaKU7v5X8_WZFEftYnokAFUGIfIuBIIjF79j0I7xU9cuqvApj5-7
M7LjZ7e3BgPy2SusHBNKYNQFAfm0dCYHsXMB0A05kvNwHSqLSKL2qs/file# [following]
--2026-08-28 03:44:46-- https://uc7aba254521e035c457b56bff3d.dl.dropboxuserconte
nt.com/cd/0/inline/DHDTBW0PaJtKmfFsk8gxq29FaKU7v5X8_WZFEftYnokAFUGIfIuBIIjF79j0
I7xU9cuqvApj5-7M7LjZ7e3BgPy2SusHBNKYNQFAfm0dCYHsXMB0A05kvNwHSqLSKL2qs/file
Resolving uc7aba254521e035c457b56bff3d.dl.dropboxusercontent.com (uc7aba254521e03
5c457b56bff3d.dl.dropboxusercontent.com)... 162.125.1.15, 2620:100:6016:15::a27d:
10f
Connecting to uc7aba254521e035c457b56bff3d.dl.dropboxusercontent.com (uc7aba25452
1e035c457b56bff3d.dl.dropboxusercontent.com)|162.125.1.15|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 686 [text/plain]
Saving to: 'mines.csv'
```

```
mines.csv          100%[=====>]          686  --.-KB/s    in 0s
```

```
2026-08-28 03:44:46 (203 MB/s) - 'mines.csv' saved [686/686]
```

```
In [18]: import pandas as pd
mines = pd.read_csv('mines.csv')
mines.head()
```

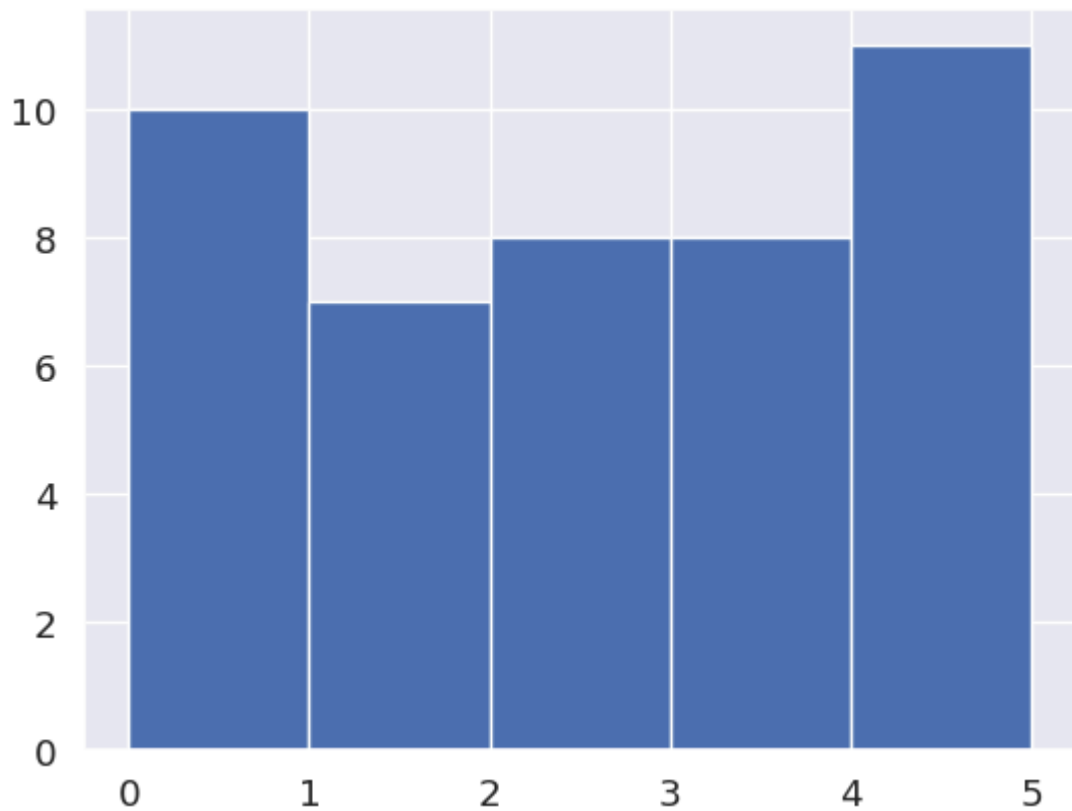
```
Out[18]:
```

	COUNT	INB	EXTRP	AHS	AGE
0	2	50	70	52	1.0
1	1	230	65	42	6.0
2	0	125	70	45	1.0
3	4	75	65	68	0.5
4	1	70	65	53	0.5

6. Visualize data distribution

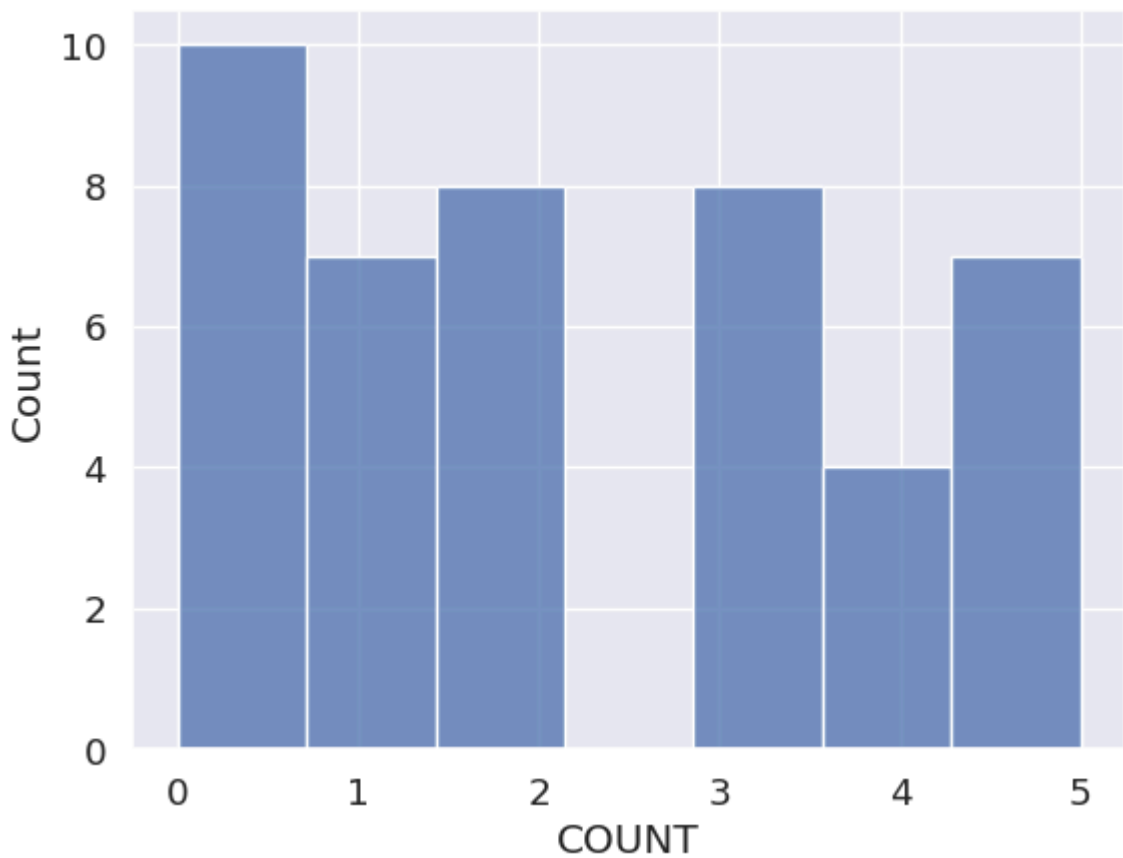
```
In [19]: mines['COUNT'].hist(bins=5)
```

```
Out[19]: <Axes: >
```



```
In [20]: import seaborn as sns  
sns.histplot(mines['COUNT'])
```

```
Out[20]: <Axes: xlabel='COUNT', ylabel='Count'>
```



7. Poisson regression

with statmodels

```
In [21]: import statsmodels.api as sm
from statsmodels.genmod.families import Poisson
```

```
In [22]: X = mines.drop(columns='COUNT')
X_prep = sm.add_constant(X)
y = mines['COUNT']
model = sm.GLM(y, X_prep, family = Poisson()).fit()
model.summary()
```

Out[22]: Generalized Linear Model Regression Results

Dep. Variable:	COUNT	No. Observations:	44
Model:	GLM	Df Residuals:	39
Model Family:	Poisson	Df Model:	4
Link Function:	Log	Scale:	1.0000
Method:	IRLS	Log-Likelihood:	-66.995
Date:	Fri, 28 Aug 2026	Deviance:	37.717
Time:	03:44:55	Pearson chi2:	35.8
No. Iterations:	5	Pseudo R-squ. (CS):	0.5713
Covariance Type:	nonrobust		

	coef	std err	z	P> z	[0.025	0.975]
const	-3.6097	1.028	-3.510	0.000	-5.625	-1.594
INB	-0.0014	0.001	-1.716	0.086	-0.003	0.000
EXTRP	0.0622	0.012	5.062	0.000	0.038	0.086
AHS	-0.0018	0.005	-0.346	0.729	-0.012	0.008
AGE	-0.0296	0.016	-1.816	0.069	-0.062	0.002

```
In [23]: y_res = model.predict(X_prep)
```

```
In [24]: from sklearn.metrics import mean_absolute_error
mean_absolute_error(y_true=y, y_pred=y_res)
```

Out[24]: 0.9307388957080918

Evaluation

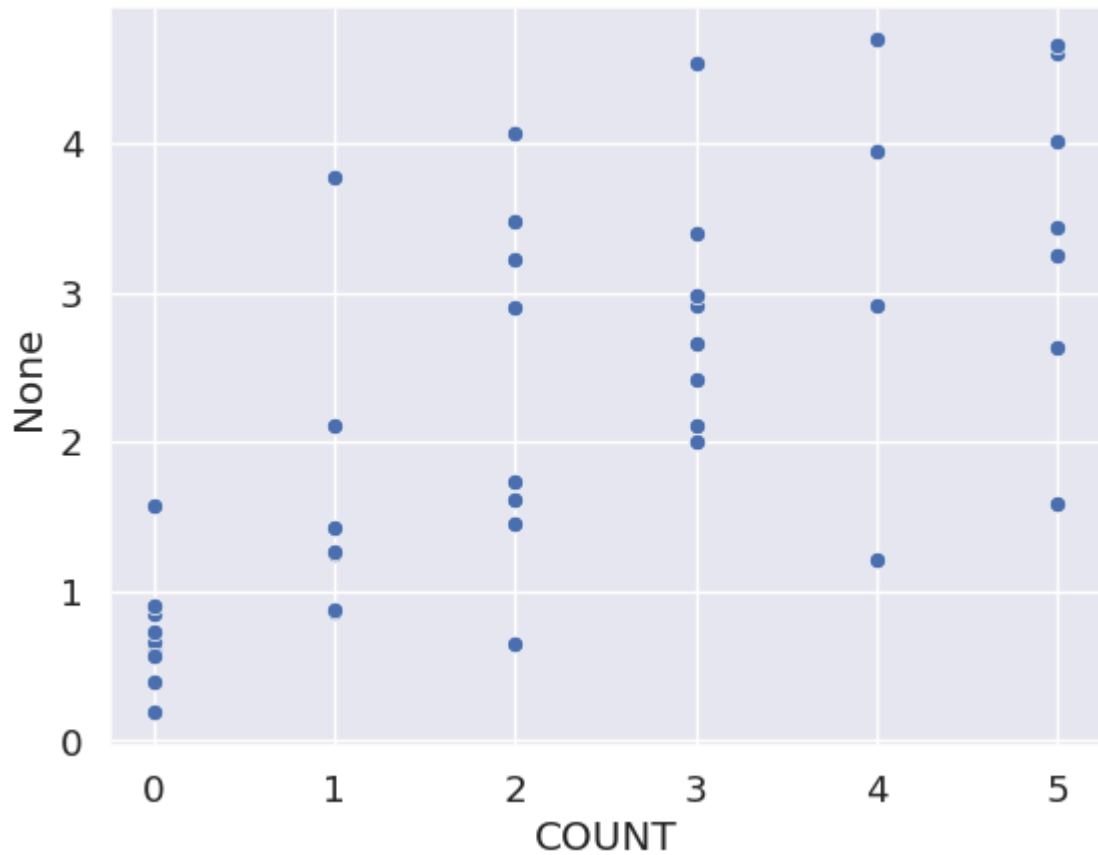
```
In [25]: from sklearn.metrics import mean_absolute_error
```

```
In [26]: mean_absolute_error(y_pred=model.predict(X_prep), y_true=y)
```

Out[26]: 0.9307388957080918

```
In [27]: sns.scatterplot(x=y, y=model.predict(X_prep))
```

Out[27]: <Axes: xlabel='COUNT', ylabel='None'>



Compare to linear regression

MAE of Poisson Regression = 0.9307

```
In [28]: X_prep = sm.add_constant(X)
lm = sm.OLS(y, X_prep).fit()
lm.summary()
```

Out[28]:

OLS Regression Results

Dep. Variable:	COUNT	R-squared:	0.530
Model:	OLS	Adj. R-squared:	0.482
Method:	Least Squares	F-statistic:	11.00
Date:	Fri, 28 Aug 2026	Prob (F-statistic):	4.55e-06
Time:	03:45:03	Log-Likelihood:	-69.941
No. Observations:	44	AIC:	149.9
Df Residuals:	39	BIC:	158.8
Df Model:	4		
Covariance Type:	nonrobust		

	coef	std err	t	P> t	[0.025	0.975]
const	-4.5797	1.379	-3.321	0.002	-7.369	-1.790
INB	-0.0020	0.001	-1.524	0.136	-0.005	0.001
EXTRP	0.1042	0.017	6.119	0.000	0.070	0.139
AHS	-0.0077	0.009	-0.901	0.373	-0.025	0.010
AGE	-0.0486	0.028	-1.725	0.092	-0.106	0.008

Omnibus:	1.311	Durbin-Watson:	2.526
Prob(Omnibus):	0.519	Jarque-Bera (JB):	1.056
Skew:	0.375	Prob(JB):	0.590
Kurtosis:	2.877	Cond. No.	1.77e+03

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The condition number is large, 1.77e+03. This might indicate that there are strong multicollinearity or other numerical problems.

```
In [29]: y_res = lm.predict(X_prep)
         mean_absolute_error(y, y_res)
```

Out[29]: 0.9366988953509331

In [30]: y_res

Out[30]:

0**0** 2.170637**1** 1.128059**2** 2.076445**3** 1.501468**4** 1.626576**5** 2.089919**6** 1.021765**7** 0.545242**8** 3.672588**9** 3.152231**10** 1.132050**11** 3.900345**12** 1.942372**13** 3.367701**14** 3.223064**15** 3.806386**16** 3.869613**17** 3.262442**18** 0.595756**19** 3.490417**20** 3.518025**21** 1.829621**22** 2.344900**23** 2.746063**24** 1.462799**25** 3.757984**26** -1.360553**27** 2.533680**28** 2.038730**29** 0.956509**30** 3.157593**31** 3.278133**32** 3.893867

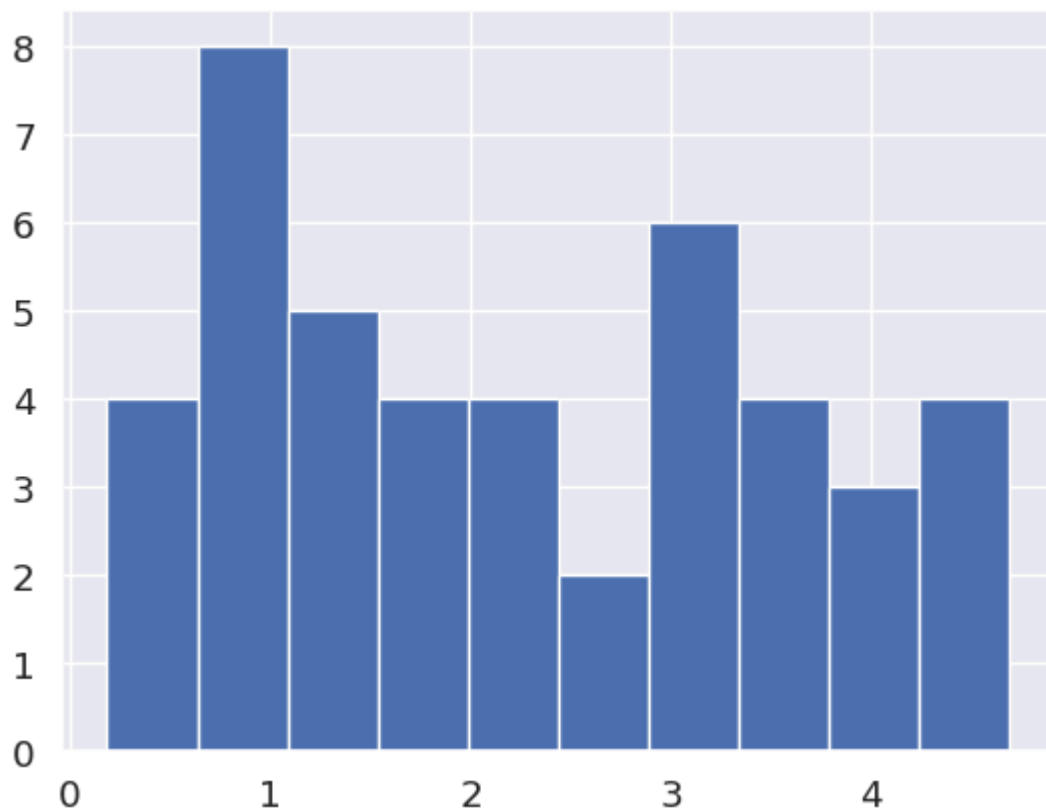
0

33	-0.244533
34	0.536512
35	3.142633
36	0.919443
37	1.008225
38	2.949301
39	3.095132
40	2.581973
41	2.894245
42	0.322657
43	3.061987

dtype: float64

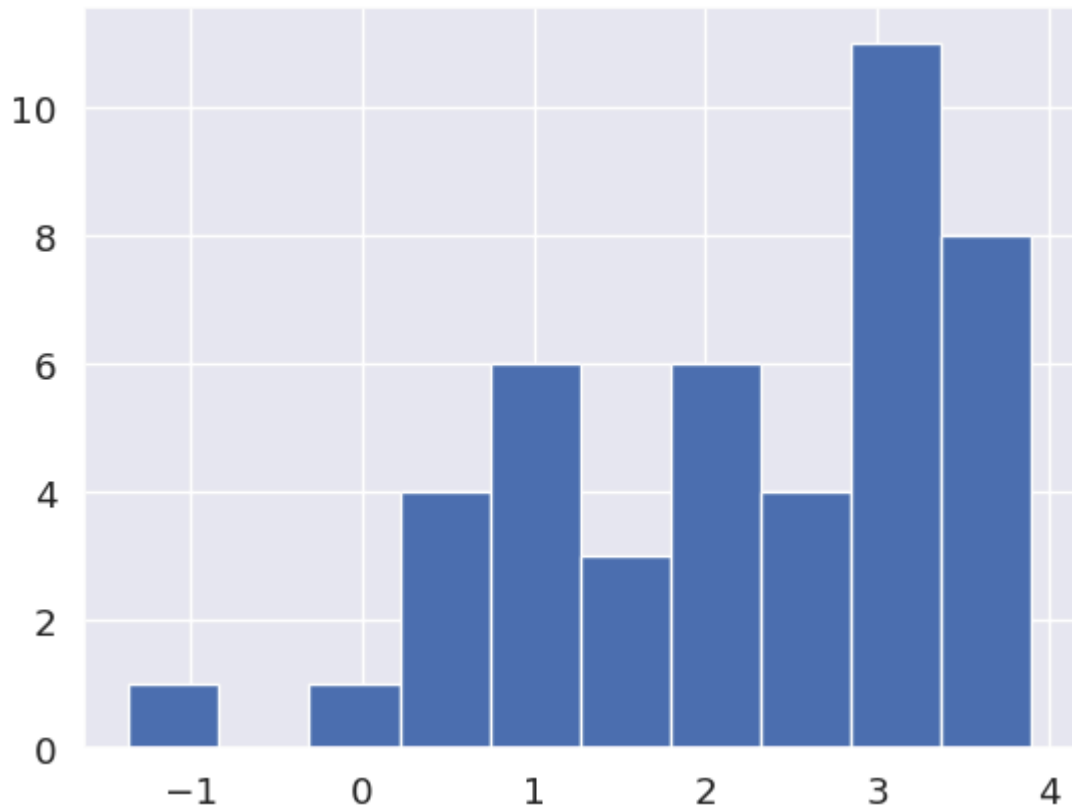
```
In [31]: model.predict(X_prep).hist()
```

```
Out[31]: <Axes: >
```



```
In [32]: lm.predict(X_prep).hist()
```

```
Out[32]: <Axes: >
```



Activity

Use the cancer.xls data and perform Poisson regression analysis

```
In [33]: !wget -O cancer.xls https://www.dropbox.com/scl/fi/liwhrsk2voczh7jmajrts/cancer.
```

```
--2026-08-28 03:45:10-- https://www.dropbox.com/scl/fi/liwhrsk2voczh7jmajrts/cancer.xls?rlkey=ws3yj709k279k7qaj9iixjw23
Resolving www.dropbox.com (www.dropbox.com)... 162.125.1.18, 2620:100:6016:18::a27d:112
Connecting to www.dropbox.com (www.dropbox.com)|162.125.1.18|:443... connected.
HTTP request sent, awaiting response... 302 Found
Location: https://uc4eba5a60348e1c3d183f81d82a.dl.dropboxusercontent.com/cd/0/inline/DHA75ocD3c6xgmi9RNEmXxFzCASgMYUj-jo2BrZxdhF_sHRTqRPVjHbRiMWCUpag-q_6WNv9KHnckoJKeXwL-bDdAY99Y1ur0sJjPQpsvzR04laHJmngK2nY54dsxPTAKk/file# [following]
--2026-08-28 03:45:10-- https://uc4eba5a60348e1c3d183f81d82a.dl.dropboxusercontent.com/cd/0/inline/DHA75ocD3c6xgmi9RNEmXxFzCASgMYUj-jo2BrZxdhF_sHRTqRPVjHbRiMWCUpag-q_6WNv9KHnckoJKeXwL-bDdAY99Y1ur0sJjPQpsvzR04laHJmngK2nY54dsxPTAKk/file
Resolving uc4eba5a60348e1c3d183f81d82a.dl.dropboxusercontent.com (uc4eba5a60348e1c3d183f81d82a.dl.dropboxusercontent.com)... 162.125.1.15, 2620:100:6016:15::a27d:10f
Connecting to uc4eba5a60348e1c3d183f81d82a.dl.dropboxusercontent.com (uc4eba5a60348e1c3d183f81d82a.dl.dropboxusercontent.com)|162.125.1.15|:443... connected.
HTTP request sent, awaiting response... 302 Found
Location: /cd/0/inline2/DHDSJBziHWltPCDap37Uie_jDQY4P74rHldjV8r4jrEk6kZxPLnTCoEXBdWew3k8zHGGgk1PBhlrZ3MTMGXA00uDjzzJVA0iUPe010B6y-I_HAUgVQOUz-DsaKeS0cvkAAaT5LxvjrbtwvHocVN5saDTSdV7CgVFCEfheqd2E2qgJWOP9Jd6H20hmoBtXfF9XxwF3PEftEPv0pe9CcuYdFYW7M1JnSVsaLaWfpTEFFAwS_10z72xTjp29DuZ572PMUUGcJZgZyi-QG3f1VKkwMki8y92Kod4eqH00oIL4q64YMiJq70zWkf6d6oZNZ0WhSTcPFTkr6-yaf4q8RssyIHhg3AHcoY9JEUbhGHYU9ofsw/file [following]
--2026-08-28 03:45:11-- https://uc4eba5a60348e1c3d183f81d82a.dl.dropboxusercontent.com/cd/0/inline2/DHDSJBziHWltPCDap37Uie_jDQY4P74rHldjV8r4jrEk6kZxPLnTCoEXBdWew3k8zHGGgk1PBhlrZ3MTMGXA00uDjzzJVA0iUPe010B6y-I_HAUgVQOUz-DsaKeS0cvkAAaT5LxvjrbtwvHocVN5saDTSdV7CgVFCEfheqd2E2qgJWOP9Jd6H20hmoBtXfF9XxwF3PEftEPv0pe9CcuYdFYW7M1JnSVsaLaWfpTEFFAwS_10z72xTjp29DuZ572PMUUGcJZgZyi-QG3f1VKkwMki8y92Kod4eqH00oIL4q64YMiJq70zWkf6d6oZNZ0WhSTcPFTkr6-yaf4q8RssyIHhg3AHcoY9JEUbhGHYU9ofsw/file
Reusing existing connection to uc4eba5a60348e1c3d183f81d82a.dl.dropboxusercontent.com:443.
HTTP request sent, awaiting response... 200 OK
Length: 35328 (34K) [application/vnd.ms-excel]
Saving to: 'cancer.xls'
```

```
cancer.xls          100%[=====>]  34.50K  ---KB/s    in 0.007s
```

```
2026-08-28 03:45:11 (4.59 MB/s) - 'cancer.xls' saved [35328/35328]
```

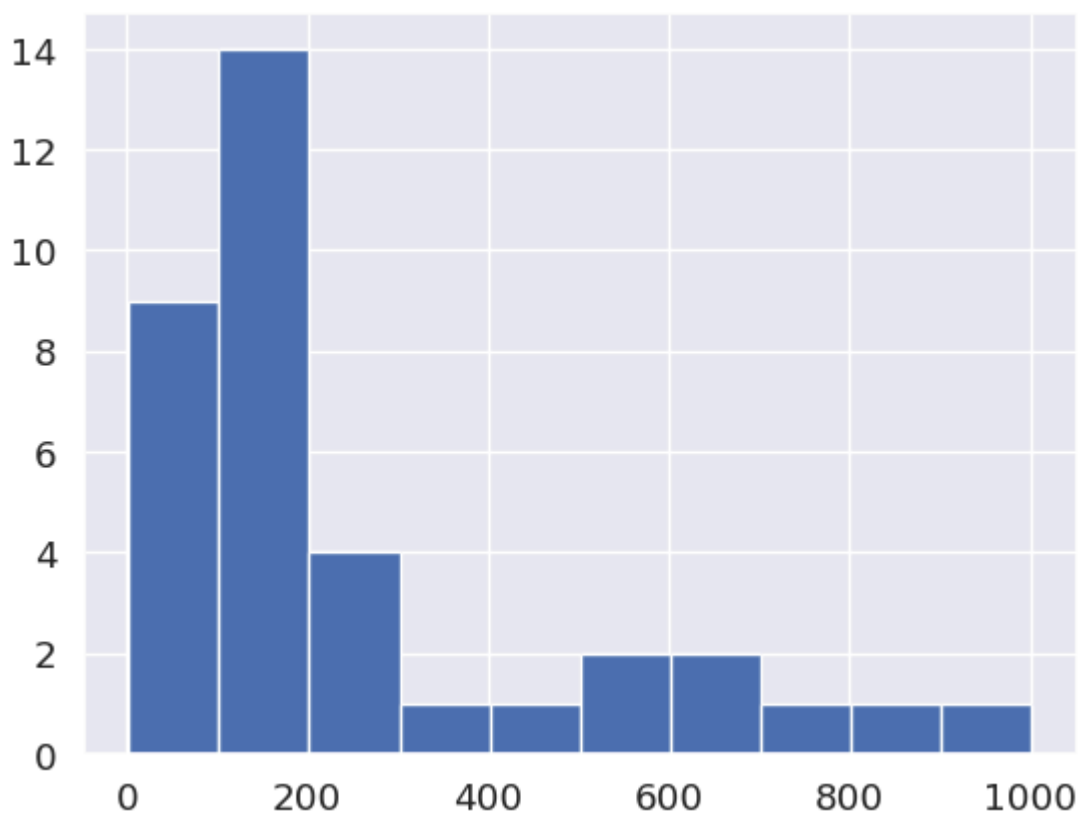
```
In [34]: # work here
cancer = pd.read_excel('cancer.xls', sheet_name='Sheet2',
                      index_col='index')
cancer.head()
```

```
Out[34]:
```

	age	smoke	pop	dead
index				
1	40-44	no	656	18
2	45-59	no	359	22
3	50-54	no	249	19
4	55-59	no	632	55
5	60-64	no	1067	117

```
In [35]: cancer['dead'].hist()
```

```
Out[35]: <Axes: >
```



```
In [36]: cancer['smoke'].value_counts()
```

```
Out[36]:
```

count	
smoke	
no	9
cigarPipeOnly	9
cigarettePlus	9
cigaretteOnly	9

dtype: int64

```
In [37]: cancer.replace({'smoke':{'no':'0_no'}}, inplace=True)
```

```
In [38]: cancer_cat = pd.get_dummies(cancer[['age','smoke']], drop_first=True)
cancer_cat.head()
```

Out[38]:

	age_45-59	age_50-54	age_55-59	age_60-64	age_65-69	age_70-74	age_75-79	age_80+	smoke_cig
index									
1	False	False	False	False	False	False	False	False	
2	True	False	False	False	False	False	False	False	
3	False	True	False	False	False	False	False	False	
4	False	False	True	False	False	False	False	False	
5	False	False	False	True	False	False	False	False	

In [39]: `cancer_prep = pd.concat([cancer_cat,cancer['pop']], axis=1)`
In [42]: `from sklearn.preprocessing import MinMaxScaler
mms = MinMaxScaler()
cancer_prep_1 = pd.DataFrame(mms.fit_transform(cancer_prep), columns=cancer_prep.columns)
cancer_prep_1.head()`

Out[42]:

	age_45-59	age_50-54	age_55-59	age_60-64	age_65-69	age_70-74	age_75-79	age_80+	smoke_cig
0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	
1	1.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	
2	0.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	
3	0.0	0.0	1.0	0.0	0.0	0.0	0.0	0.0	
4	0.0	0.0	0.0	1.0	0.0	0.0	0.0	0.0	

In [43]: `cancer['dead']`

Out[43]:

dead	
index	
1	18
2	22
3	19
4	55
5	117
6	170
7	179
8	120
9	120
10	2
11	4
12	3
13	38
14	113
15	173
16	212
17	243
18	253
19	149
20	169
21	193
22	576
23	1001
24	901
25	613
26	337
27	189
28	124
29	140
30	187
31	514
32	778

dead	
index	
33	689
34	432
35	214
36	63

dtype: int64

```
In [44]: model = sm.GLM(cancer['dead'].reset_index(drop=True),  
                        sm.add_constant(cancer_prep_1),  
                        family = Poisson()).fit()  
model.summary()
```


Out[44]:

Generalized Linear Model Regression Results

Dep. Variable:	dead	No. Observations:	36
Model:	GLM	Df Residuals:	23
Model Family:	Poisson	Df Model:	12
Link Function:	Log	Scale:	1.0000
Method:	IRLS	Log-Likelihood:	-412.46
Date:	Fri, 28 Aug 2026	Deviance:	584.90
Time:	04:22:28	Pearson chi2:	508.
No. Iterations:	6	Pseudo R-squ. (CS):	1.000
Covariance Type:	nonrobust		

	coef	std err	z	P> z	[0.025	0.975]
const	2.6013	0.075	34.887	0.000	2.455	2.747
age_45-59	0.6242	0.083	7.556	0.000	0.462	0.786
age_50-54	1.0038	0.082	12.256	0.000	0.843	1.164
age_55-59	1.3818	0.065	21.174	0.000	1.254	1.510
age_60-64	1.5012	0.065	23.044	0.000	1.374	1.629
age_65-69	2.1366	0.064	33.517	0.000	2.012	2.262
age_70-74	2.3557	0.072	32.939	0.000	2.215	2.496
age_75-79	2.2041	0.079	27.817	0.000	2.049	2.359
age_80+	1.9506	0.085	23.084	0.000	1.785	2.116
smoke_cigarPipeOnly	0.2114	0.047	4.520	0.000	0.120	0.303
smoke_cigaretteOnly	0.7818	0.044	17.868	0.000	0.696	0.868
smoke_cigarettePlus	0.5030	0.057	8.821	0.000	0.391	0.615
pop	2.4840	0.099	25.099	0.000	2.290	2.678

In [45]: `model.predict(sm.add_constant(cancer_prep_1))`

Out[45]:

0

0	17.014227
1	28.061053
2	39.176018
3	67.078251
4	90.624064
5	159.360220
6	180.317513
7	136.334058
8	102.034593
9	16.983526
10	31.167641
11	45.442675
12	74.349106
13	102.094869
14	201.189285
15	237.742306
16	191.360037
17	140.670556
18	141.692760
19	141.412588
20	150.344691
21	600.931821
22	1199.219114
23	914.734227
24	526.974283
25	278.894174
26	173.796342
27	117.309486
28	134.358717
29	167.036615
30	440.640822
31	617.061953
32	657.716267

0

33	490.965897
34	307.411732
35	208.498509

dtype: float64

8. Survival analysis

For this example, we will be investigating the lifetimes of political leaders around the world. A political leader, in this case, is defined by a single individual's time in office who controls the ruling regime. This political leader could be an elected president, unelected dictator, monarch, etc. The birth event is the start of the individual's tenure, and the death event is the retirement of the individual. Censoring can occur if they are a) still in offices at the time of dataset compilation (2008), or b) die while in power (this includes assassinations).

For example, the Bush regime began in 2000 and officially ended in 2008 upon his retirement, thus this regime's lifespan was eight years, and there was a "death" event observed. On the other hand, the JFK regime lasted 2 years, from 1961 and 1963, and the regime's official death event was not observed – JFK died before his official retirement.

In [46]: `!pip install lifelines`

Collecting lifelines

Downloading lifelines-0.30.3-py3-none-any.whl.metadata (3.5 kB)

Requirement already satisfied: numpy>=1.14.0 in /usr/local/lib/python3.13/dist-packages (from lifelines) (2.1.3)

Requirement already satisfied: scipy>=1.7.0 in /usr/local/lib/python3.13/dist-packages (from lifelines) (1.16.3)

Requirement already satisfied: pandas<3.0,>=2.1 in /usr/local/lib/python3.13/dist-packages (from lifelines) (2.2.3)

Requirement already satisfied: matplotlib>=3.0 in /usr/local/lib/python3.13/dist-packages (from lifelines) (3.10.0)

Requirement already satisfied: autograd>=1.5 in /usr/local/lib/python3.13/dist-packages (from lifelines) (1.9.1)

Collecting autograd-gamma>=0.3 (from lifelines)

Downloading autograd-gamma-0.5.0.tar.gz (4.0 kB)

Preparing metadata (setup.py) ... done

Collecting formulaic>=0.2.2 (from lifelines)

Downloading formulaic-1.2.2-py3-none-any.whl.metadata (7.0 kB)

Collecting interface-meta>=1.2.0 (from formulaic>=0.2.2->lifelines)

Downloading interface_meta-2.0.1-py3-none-any.whl.metadata (6.4 kB)

Requirement already satisfied: narwhals>=1.17 in /usr/local/lib/python3.13/dist-packages (from formulaic>=0.2.2->lifelines) (2.25.0)

Requirement already satisfied: typing-extensions>=4.2.0 in /usr/local/lib/python3.13/dist-packages (from formulaic>=0.2.2->lifelines) (4.16.0)

Requirement already satisfied: wrapt>=1.17.0rc1 in /usr/local/lib/python3.13/dist-packages (from formulaic>=0.2.2->lifelines) (2.3.0)

Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.13/dist-packages (from matplotlib>=3.0->lifelines) (1.3.3)

Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.13/dist-packages (from matplotlib>=3.0->lifelines) (0.12.1)

Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.13/dist-packages (from matplotlib>=3.0->lifelines) (4.63.0)

Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.13/dist-packages (from matplotlib>=3.0->lifelines) (1.5.0)

Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.13/dist-packages (from matplotlib>=3.0->lifelines) (26.3)

Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.13/dist-packages (from matplotlib>=3.0->lifelines) (11.3.0)

Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.13/dist-packages (from matplotlib>=3.0->lifelines) (3.3.2)

Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.13/dist-packages (from matplotlib>=3.0->lifelines) (2.9.0.post0)

Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.13/dist-packages (from pandas<3.0,>=2.1->lifelines) (2025.2)

Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.13/dist-packages (from pandas<3.0,>=2.1->lifelines) (2026.3)

Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.13/dist-packages (from python-dateutil>=2.7->matplotlib>=3.0->lifelines) (1.17.0)

Downloading lifelines-0.30.3-py3-none-any.whl (409 kB)

409.1/409.1 kB 11.0 MB/s eta 0:00:00

Downloading formulaic-1.2.2-py3-none-any.whl (118 kB)

118.9/118.9 kB 9.3 MB/s eta 0:00:00

Downloading interface_meta-2.0.1-py3-none-any.whl (15 kB)

Building wheels for collected packages: autograd-gamma

Building wheel for autograd-gamma (setup.py) ... done

Created wheel for autograd-gamma: filename=autograd_gamma-0.5.0-py3-none-any.whl size=4030 sha256=ec8ca5968f6e5e54a3dfab1cf11d78290c2d8dd60204b05ee7a7457590fd240e

Stored in directory: /root/.cache/pip/wheels/7e/16/46/9477f188924292d3bf1fb8fb42844201591abfc19b7ba6d868

Successfully built autograd-gamma

Installing collected packages: interface-meta, autograd-gamma, formulaic, lifelines

Successfully installed autograd-gamma-0.5.0 formulaic-1.2.2 interface-meta-2.0.1 lifelines-0.30.3

In [47]: `from lifelines.datasets import load_dd`

```
data = load_dd()
data.head()
```

Out[47]:

	ctryname	cowcode2	politycode	un_region_name	un_continent_name	ehead
--	----------	----------	------------	----------------	-------------------	-------

0	Afghanistan	700	700.0	Southern Asia	Asia	Mohammad Zahir Shah
---	-------------	-----	-------	---------------	------	---------------------

1	Afghanistan	700	700.0	Southern Asia	Asia	Sardar Mohammad Daoud
---	-------------	-----	-------	---------------	------	-----------------------

2	Afghanistan	700	700.0	Southern Asia	Asia	Mohammad Zahir Shah
---	-------------	-----	-------	---------------	------	---------------------

3	Afghanistan	700	700.0	Southern Asia	Asia	Sardar Mohammad Daoud
---	-------------	-----	-------	---------------	------	-----------------------

4	Afghanistan	700	700.0	Southern Asia	Asia	Nur Mohammad Taraki
---	-------------	-----	-------	---------------	------	---------------------



In [48]: `data.shape`

Out[48]: (1808, 12)

Let's take a look at our country.

In [49]: `data[data['ctryname'] == 'Thailand']`

Out[49]:

	ctryname	cowcode2	politycode	un_region_name	un_continent_name	
1602	Thailand	800	800.0	South-Eastern Asia	Asia	Thamrong
1603	Thailand	800	800.0	South-Eastern Asia	Asia	Phibuls
1604	Thailand	800	800.0	South-Eastern Asia	Asia	Thanom K
1605	Thailand	800	800.0	South-Eastern Asia	Asia	Sar
1606	Thailand	800	800.0	South-Eastern Asia	Asia	Thanom K
1607	Thailand	800	800.0	South-Eastern Asia	Asia	Sanya T
1608	Thailand	800	800.0	South-Eastern Asia	Asia	Ku
1609	Thailand	800	800.0	South-Eastern Asia	Asia	
1610	Thailand	800	800.0	South-Eastern Asia	Asia	C
1611	Thailand	800	800.0	South-Eastern Asia	Asia	C
1612	Thailand	800	800.0	South-Eastern Asia	Asia	Prem T
1613	Thailand	800	800.0	South-Eastern Asia	Asia	Cl
1614	Thailand	800	800.0	South-Eastern Asia	Asia	
1615	Thailand	800	800.0	South-Eastern Asia	Asia	Chu
1616	Thailand	800	800.0	South-Eastern Asia	Asia	Banharn S
1617	Thailand	800	800.0	South-Eastern Asia	Asia	Yor
1618	Thailand	800	800.0	South-Eastern Asia	Asia	Chu
1619	Thailand	800	800.0	South-Eastern Asia	Asia	Thaksin
1620	Thailand	800	800.0	South-Eastern Asia	Asia	Surayud
1621	Thailand	800	800.0	South-Eastern Asia	Asia	Abhi



```
In [50]: from lifelines import KaplanMeierFitter
kmf = KaplanMeierFitter()

T = data["duration"]
E = data["observed"]

kmf.fit(T, event_observed=E)
```

```
Out[50]: <lifelines.KaplanMeierFitter:"KM_estimate", fitted with 1808 total observations, 340 right-censored observations>
```

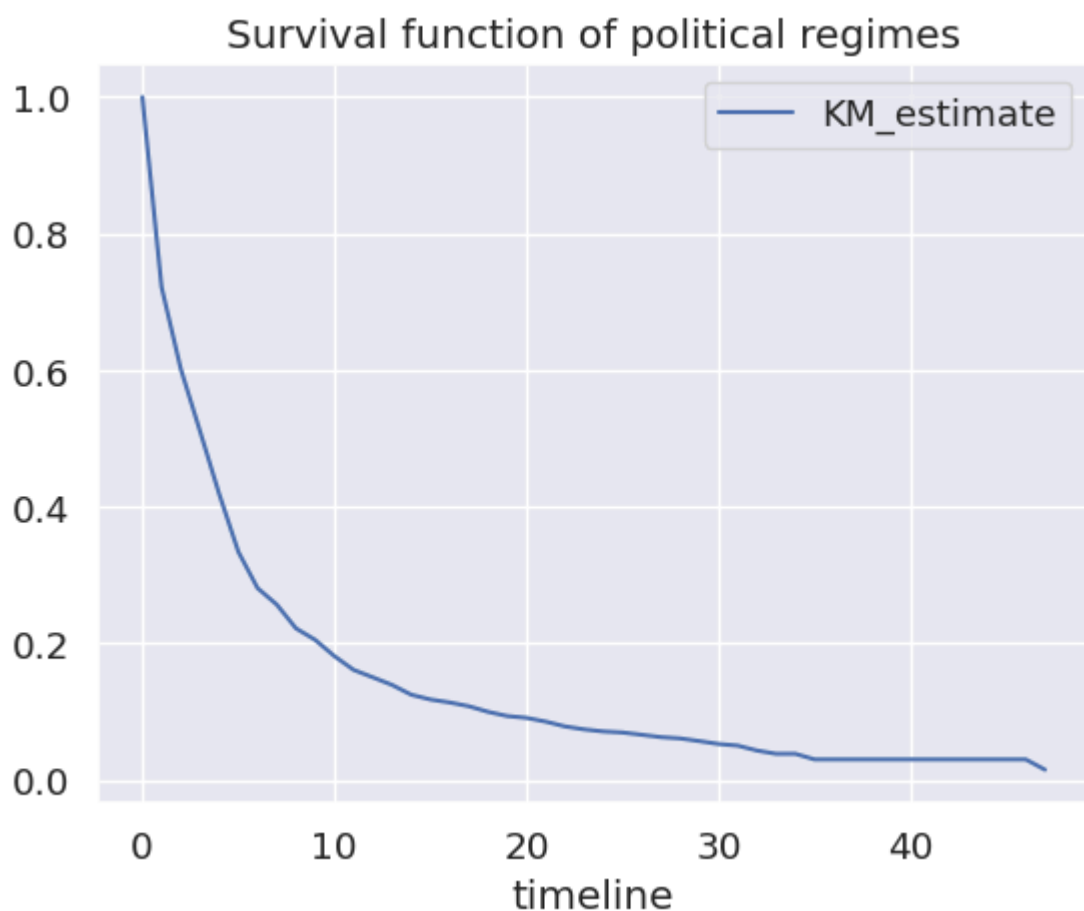
```
In [51]: kmf.survival_function_at_times(10)
```

```
Out[51]:
```

	KM_estimate
10	0.18135

dtype: float64

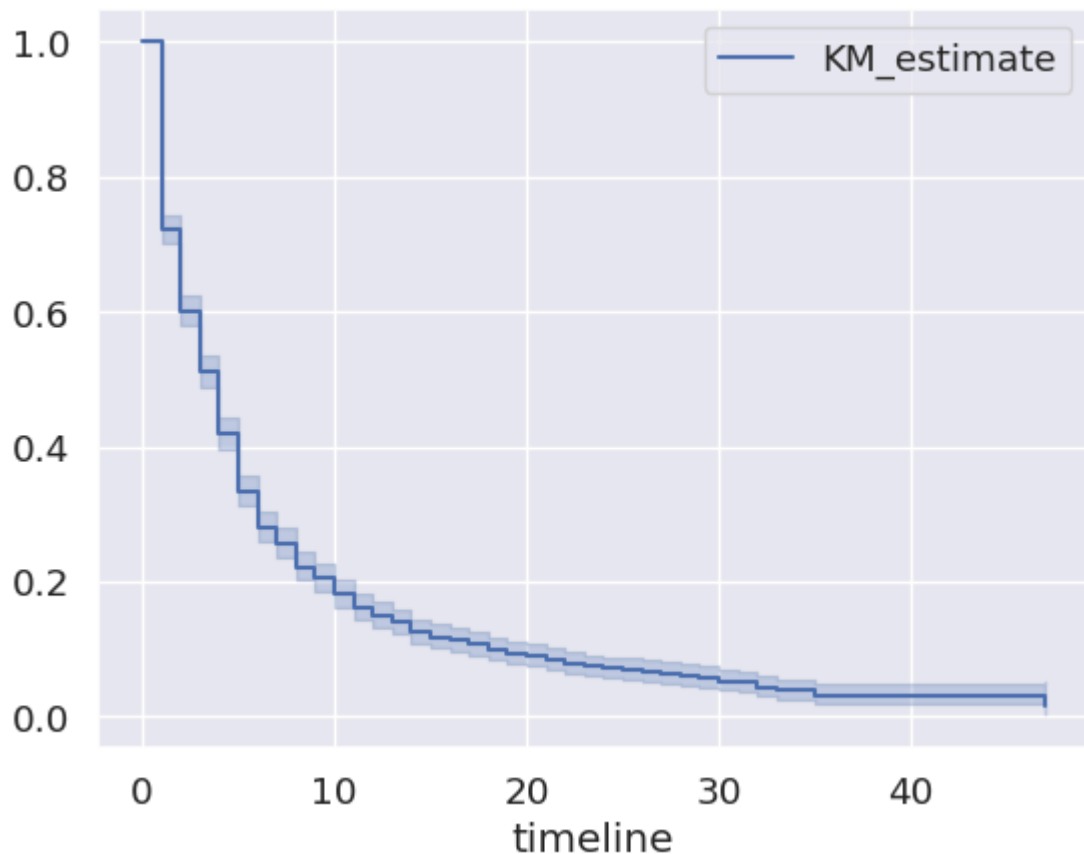
```
In [52]: import matplotlib.pyplot as plt
kmf.survival_function_.plot()
plt.title('Survival function of political regimes');
```



How do we interpret this? The y-axis represents the probability a leader is still around after t years, where t years is on the x-axis. We see that very few leaders make it past 20 years in office.

```
In [53]: kmf.plot()
```

Out[53]: <Axes: xlabel='timeline'>



The median time in office, which defines the point in time where on average 1/2 of the population has expired, is a property:

In [54]: `kmf.median_survival_time_`

Out[54]: `np.float64(4.0)`

Interesting that it is only four years. That means, around the world, elected leaders have a 50% chance of cessation in four years or less!

Plot by segment (regimes)

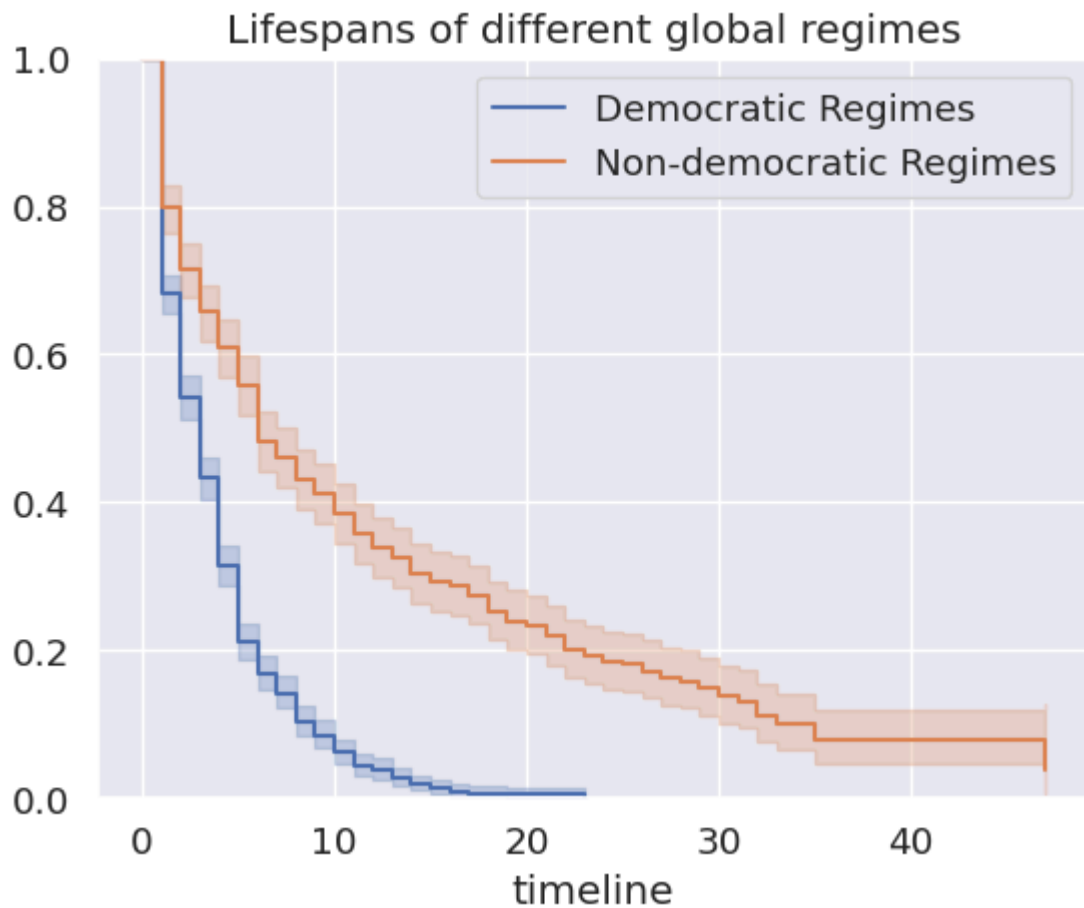
Let's segment on democratic regimes vs non-democratic regimes. Calling plot on either the estimate itself or the fitter object will return an axis object, that can be used for plotting further estimates:

```
In [55]: ax = plt.subplot(111)

dem = (data["democracy"] == "Democracy")

kmf.fit(T[dem], event_observed=E[dem], label="Democratic Regimes")
kmf.plot(ax=ax)
kmf.fit(T[~dem], event_observed=E[~dem], label="Non-democratic Regimes")
kmf.plot(ax=ax)

plt.ylim(0, 1);
plt.title("Lifespans of different global regimes");
```

Mathematical Notations

Survival function

We will assume for now that T is a continuous random variable with probability density function (p.d.f.)

$$f(t)$$

and cumulative distribution function (c.d.f.)

$$F(t) = \Pr(T < t),$$

giving the probability that the event has occurred by duration t .

$$S(t) = \Pr\{T \geq t\} = 1 - F(t) = \int_t^{\infty} f(x)dx,$$

Hazard function

An alternative characterization of the distribution of T is given by the hazard function, or instantaneous rate of occurrence of the event, defined as

$$\lambda(t) = \lim_{dt \rightarrow 0} \frac{\Pr\{t \leq T < t + dt | T \geq t\}}{dt}.$$

9. Cox's proportional hazard model

The idea behind Cox's proportional hazard model model is that the log-hazard of an individual is a linear function of their static covariates and a population-level baseline hazard that changes over time. Mathematically:

$$\underbrace{h(t|x)}_{\text{hazard}} = \underbrace{b_0(t)}_{\text{baseline hazard}} \underbrace{\exp\left(\sum_{i=1}^n b_i(x_i - \bar{x}_i)\right)}_{\text{log-partial hazard}} \underbrace{\hspace{1cm}}_{\text{partial hazard}}$$

```
In [56]: from lifelines import CoxPHFitter
from lifelines.datasets import load_rossi

rossi_dataset = load_rossi()
rossi_dataset.head()
```

```
Out[56]:
```

	week	arrest	fin	age	race	wexp	mar	paro	prio
0	20	1	0	27	1	0	0	1	3
1	17	1	0	18	1	0	0	1	8
2	25	1	0	19	0	1	0	1	13
3	52	0	1	23	1	1	1	1	1
4	52	0	0	19	0	1	0	1	3

This data set is originally from Rossi et al. (1980), and is used as an example in Allison (1995). The data pertain to 432 convicts who were released from Maryland state prisons in the 1970s and who were followed up for one year after release. Half the released convicts were assigned at random to an experimental treatment in which they were given financial aid; half did not receive aid.

- week: week of first arrest after release or censoring; all censored observations are censored at 52 weeks.
- arrest: 1 if arrested, 0 if not arrested.
- fin: financial aid: no yes.
- age: in years at time of release.
- race: black or other.
- wexp: full-time work experience before incarceration: no or yes.
- mar: marital status at time of release: married or not married.
- paro: released on parole? no or yes.

- prio: number of convictions prior to current incarceration.

```
In [57]: cph = CoxPHFitter()  
cph.fit(rossi_dataset, duration_col='week', event_col='arrest', show_progress=True)  
  
cph.print_summary()
```

Iteration 1: norm_delta = 5.09e-01, step_size = 0.9500, log_lik = -675.38063, newton_decrement = 1.68e+01, seconds_since_start = 0.1

Iteration 2: norm_delta = 1.39e-01, step_size = 0.9500, log_lik = -659.79004, newton_decrement = 9.92e-01, seconds_since_start = 0.1

Iteration 3: norm_delta = 1.80e-02, step_size = 0.9500, log_lik = -658.76197, newton_decrement = 1.42e-02, seconds_since_start = 0.1

Iteration 4: norm_delta = 1.83e-04, step_size = 1.0000, log_lik = -658.74766, newton_decrement = 1.32e-06, seconds_since_start = 0.2

Iteration 5: norm_delta = 1.97e-08, step_size = 1.0000, log_lik = -658.74766, newton_decrement = 1.34e-14, seconds_since_start = 0.2

Convergence success after 5 iterations.

model	lifelines.CoxPHFitter									
duration col	'week'									
event col	'arrest'									
baseline estimation	breslow									
number of observations	432									
number of events observed	114									
partial log-likelihood	-658.75									
time fit was run	2026-08-28 04:23:06 UTC									

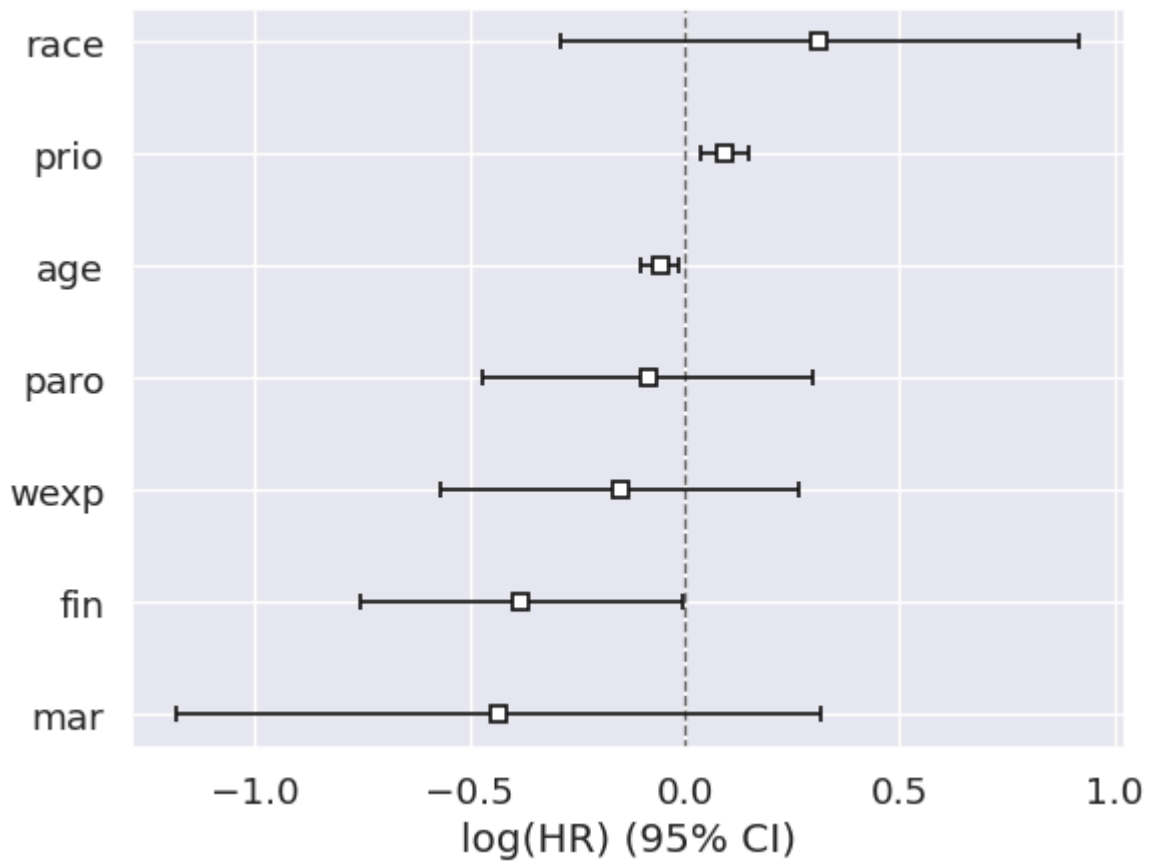
	coef	exp(coef)	se(coef)	coef lower 95%	coef upper 95%	exp(coef) lower 95%	exp(coef) upper 95%	cmp to	z	p
fin	-0.38	0.68	0.19	-0.75	-0.00	0.47	1.00	0.00	-1.98	0.05
age	-0.06	0.94	0.02	-0.10	-0.01	0.90	0.99	0.00	-2.61	0.01
race	0.31	1.37	0.31	-0.29	0.92	0.75	2.50	0.00	1.02	0.31
wexp	-0.15	0.86	0.21	-0.57	0.27	0.57	1.30	0.00	-0.71	0.48
mar	-0.43	0.65	0.38	-1.18	0.31	0.31	1.37	0.00	-1.14	0.26
paro	-0.08	0.92	0.20	-0.47	0.30	0.63	1.35	0.00	-0.43	0.66
prio	0.09	1.10	0.03	0.04	0.15	1.04	1.16	0.00	3.19	<0.005

Concordance	0.64
Partial AIC	1331.50
log-likelihood ratio test	33.27 on 7 df
-log2(p) of ll-ratio test	15.37



In [58]: `cph.plot()`

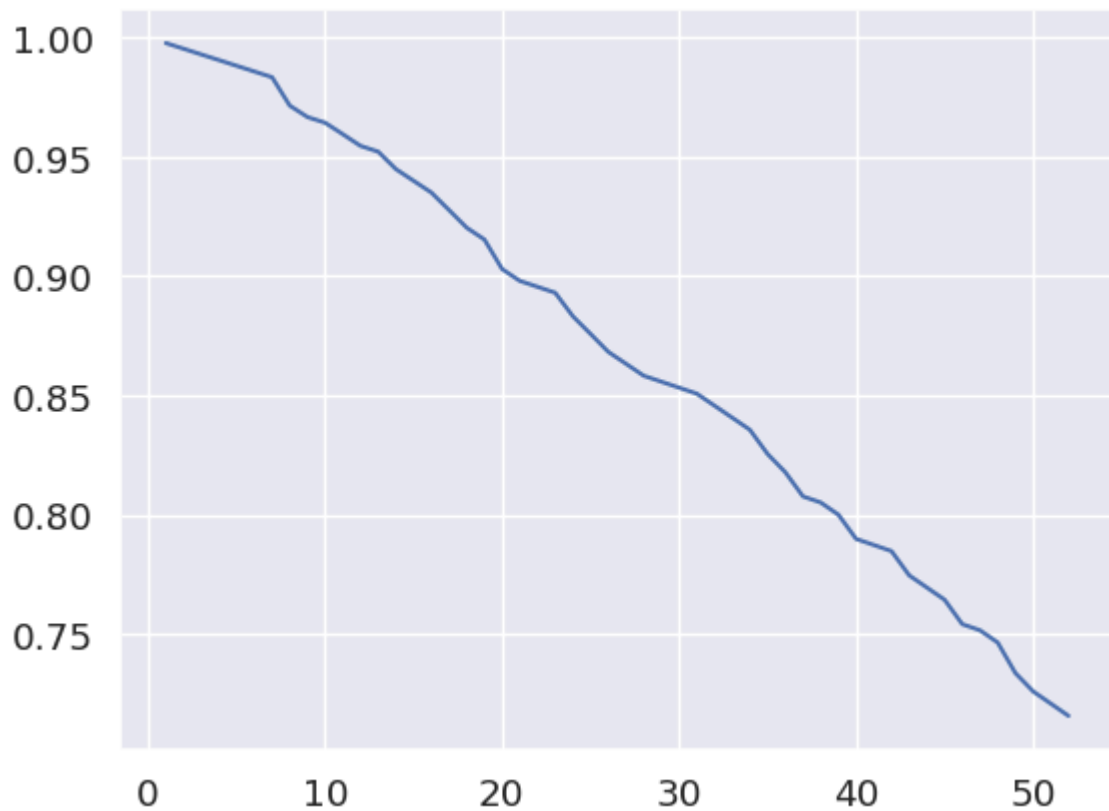
Out[58]: `<Axes: xlabel='log(HR) (95% CI)'>`



```
In [59]: X = rossi_dataset
```

```
plt.plot(cph.predict_survival_function(X[0:1]))
```

```
Out[59]: [<matplotlib.lines.Line2D at 0x7895e7d17390>]
```



Activity - Estimating Lifetime

- Use Telco Churn dataset and build a Kaplan Meier survival analysis.
- Compare lifetime between different payment methods.

```
In [60]: df = pd.read_csv('Telco+Churn.csv')
```

```
In [61]: df['Churn_Event'] = df['Churn'].apply(lambda x: 1 if x == 'Yes' else 0)
```

แปลงข้อมูลคอลัมน์ Churn จากข้อความให้เป็นตัวเลข (1 = ยกเลิก, 0 = ยังใช้งาน) เพื่อให้โมเดลสามารถคำนวณสถานะเหตุการณ์ได้

```
In [62]: kmf = KaplanMeierFitter()
payment_methods = df['PaymentMethod'].unique()

plt.figure(figsize=(10, 6))
```

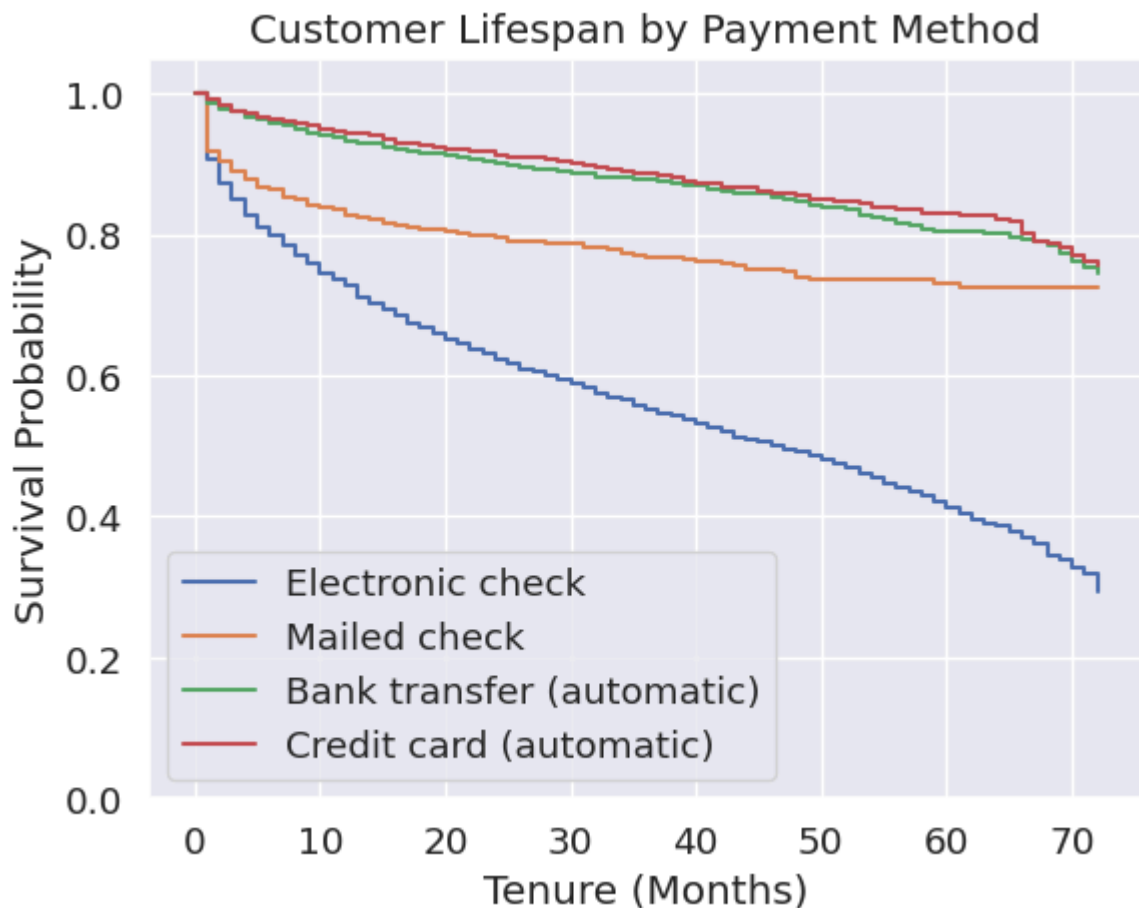
```
Out[62]: <Figure size 1000x600 with 0 Axes>
```

```
<Figure size 1000x600 with 0 Axes>
```

```
In [67]: for method in payment_methods:
    mask = df['PaymentMethod'] == method
    T = df['tenure'][mask]
    E = df['Churn_Event'][mask]

    kmf.fit(T, event_observed=E, label=method)
    kmf.plot(ci_show=False)

    plt.title('Customer Lifespan by Payment Method')
plt.xlabel('Tenure (Months)')
plt.ylabel('Survival Probability')
plt.ylim(0, 1.05)
plt.show()
```



วนลูปดึงข้อมูล Tenure และสถานะการยกเลิกแยกตามแต่ละ Payment Method และนำข้อมูลเข้าโมเดลเพื่อวาดกราฟ Survival Probability โดยปิดแสดง Confidence Interval เพื่อให้กราฟไม่ทับกัน

```
In [66]: print("Median Survival Time (Months)")
for method in payment_methods:
    mask = df['PaymentMethod'] == method
    kmf.fit(df['tenure'][mask], event_observed=df['Churn_Event'][mask])
    median_time = kmf.median_survival_time_
    print(f"{method}: {median_time}")
```

```
Median Survival Time (Months)
Electronic check: 47.0
Mailed check: inf
Bank transfer (automatic): inf
Credit card (automatic): inf
```

คำนวณ Median Survival Time เพื่อหาว่าแต่ละกลุ่มมีจำนวนลูกค้าลดลงเหลือ 50% เมื่อผ่านไปกี่เดือน

สรุปผลลัพธ์ที่ได้จะเป็น Electronic check: ลูกค้ากลุ่มนี้มีความเสี่ยงยกเลิกบริการสูงสุด เส้นกราฟตกลงเร็วที่สุดและมีค่า Median อยู่ที่ 47.0 เดือน

วิธีชำระเงินอื่นๆ (Bank transfer, Credit card, Mailed check): มีอัตราการรักษากลุ่มลูกค้าได้สูงมาก เส้นกราฟไม่เคยตกลงมาต่ำกว่าระดับ 50% ตลอด 72 เดือนของการเก็บข้อมูล

การชำระเงินด้วยระบบอัตโนมัติมีความสัมพันธ์กับอายุการใช้งานที่ยาวนานที่สุดและมีอัตราการยกเลิกต่ำที่สุด